

Middleware Engineering

"DEZSYS_GK_HELLOWORLD_GRPC"

Aufgabenstellung

Fragen

What is gRPC and why does it work across languages and platforms?

gRPC is an open-source, high-performance Remote Procedure Call (RPC) framework created originally by Google and now maintained by the Cloud Native Computing Foundation (CNCF). It allows a client application to directly call methods on a server application on another machine as if it were a local object/method call.

It works across languages because it uses **Protocol Buffers (.proto files)**, which are language-independent. The `.proto` file defines the messages and services, and gRPC automatically generates code for many languages (like Java, Python, Go, etc.). It also uses **HTTP/2**, which supports fast and efficient communication.

Describe the RPC life cycle starting with the RPC client

- The **client** calls a method on a local **stub** (generated from the `.proto` file).
- The stub **sends the request** (serialized) to the **server** via HTTP/2.
- The **server** receives and **deserializes** the message.
- The server **executes** the method and **sends a response** back.
- The client **receives and returns** the response to the application.
- Completion / termination: The RPC connection is completed (for unary calls) or remains open in case of streaming.
- Error handling / metadata: Status codes, metadata, deadlines, or errors are returned to the client.

Describe the workflow of Protocol Buffers?

- Create a `.proto` file defining messages and services.

- Use the **protoc compiler** to generate source code for your language.
- Your app uses the generated classes to **send and receive** data (serialization and deserialization).

define schema → compile → generate code → serialize/deserialize messages → use in transport/rpc workflows.

Describe the workflow of Protocol Buffers?

- Very **fast and small** (binary format).
- **Works in many languages**.
- Has a **clear structure** (schema).
- **Easy to update** (add new fields without breaking old code).
- **The tooling** generates classes, methods, serializers and deserializers for you, reducing boilerplate and potential for errors.

When is the use of protocol not recommended?

- When you need **human-readable data** (use JSON or XML instead).
- When you're dealing with **very large** data sets (messages much larger than a few megabytes), streaming large arrays/matrices, or scientific/engineering use cases where formats optimized for large numeric arrays may be better.
- When the **data format changes often** or is not structured.
- When you're in environments that require **an open standard mandated** by regulatory/law bodies and Protobuf is not formally standardized by an external standards body (depending on your context)
- When you need a **self-describing format** (Protobuf needs a schema).

List 3 different data types that can be used with protocol buffers?

- `int32` : integer number
- `string` : text
- `bool` : true/false; Boolean

HelloWorldApplication

Dependencies

- **Dependencies** are external libraries or frameworks your project needs to make certain features work. They are declared in Gradle in the `dependencies` block.

```
implementation "io.grpc:grpc-netty:${grpcVersion}" // Netzwerk/HTTP2
implementation "io.grpc:grpc-protobuf:${grpcVersion}" // Protobuf Nachrichten
implementation "io.grpc:grpc-stub:${grpcVersion}" // Client/Server Stubs
```

- **Java 9+ workaround:** adds missing annotations.
- **Test:** `junit` for unit testing.
- **Practice:** Gradle automatically downloads and includes all libraries, no manual setup needed.

Proto

- **service HelloWorldService** → defines the gRPC service.
- **rpc hello(HelloRequest) returns (HelloResponse)** → `hello` method that receives a `HelloRequest` and returns a `HelloResponse`.
- **message HelloRequest** → contains `firstname` and `lastname`.
- **message HelloResponse** → contains `text` (the server's response)
- The `.proto` file defines the interface between client and server.
- gRPC automatically generates Java classes for client and server from it.

```
syntax = "proto3";

service HelloWorldService {
  rpc hello(HelloRequest) returns (HelloResponse) {}
}

message HelloRequest {
  string firstname = 1;
  string lastname = 2;
}

message HelloResponse {
  string text = 1;
}
```

HelloWorldServer

The **gRPC server** waits for client requests and runs the defined service methods. It listens on a specific port (here: **50051**) and sends back responses. The server stays active until it's stopped manually.

```

private static final int PORT = 50051;
private Server server;

public void start() throws IOException {
    server = ServerBuilder.forPort(PORT)
        .addService(new HelloWorldServiceImpl())
        .build()
        .start();
}

public void blockUntilShutdown() throws InterruptedException {
    if (server == null) {
        return;
    }
    server.awaitTermination();
}

public static void main(String[] args) throws InterruptedException, IOException {
    HelloWorldServer server = new HelloWorldServer();
    server.start();
    server.blockUntilShutdown();
}
}

```

- Starts a gRPC server on port 50051.
- `ServerBuilder.forPort(PORT)` → erstellt den Server auf diesem Port.
- `addService(new HelloWorldServiceImpl())` → fügt den gRPC-Service hinzu.
- `build().start()` → startet den Server.
- `blockUntilShutdown()` → hält den Server am Laufen, bis er manuell beendet wird.

HelloWorldClient

- Connects to the gRPC server on **localhost:50051**.
- Sends a **HelloRequest** with `firstname` and `lastname`.
- Receives a **HelloResponse** from the server and prints the message.
- Closes the connection after the response.

```
ManagedChannel channel = ManagedChannelBuilder.forAddress("localhost", 50051)
    .usePlaintext()
    .build();

HelloWorldServiceGrpc.HelloWorldServiceBlockingStub stub =
HelloWorldServiceGrpc.newBlockingStub(channel);

Hello.HelloResponse helloResponse = stub.hello(Hello.HelloRequest.newBuilder()
    .setFirstname(firstname)
    .setLastname(lastname)
    .build());
System.out.println( helloResponse.getText() );
channel.shutdown();
```

- **ManagedChannelBuilder.forAddress("localhost", 50051)** → erstellt die Verbindung zum Server (Adresse + Port).
- **usePlaintext()** → keine Verschlüsselung (nur für Tests).
- **build()** → baut den Channel (die Verbindung).
- **HelloWorldServiceGrpc.newBlockingStub(channel)** → erstellt den Client, um Server-Methoden aufzurufen.
- **stub.hello(...)** → ruft die `hello`-Methode auf dem Server auf.
- **channel.shutdown()** → beendet die Verbindung sauber.

HelloWorldServiceImpl

This class is the **server logic**.

It defines what happens when the client calls the `hello()` method — the server processes the data and sends back a response message.

```

@Override
    public void hello( Hello.HelloRequest request, StreamObserver<Hello.HelloResponse>
responseObserver) {

        System.out.println("Handling hello endpoint: " + request.toString());

        String text = "Hello World, " + request.getFirstname() + " " +
request.getLastname();
        Hello.HelloResponse response =
Hello.HelloResponse.newBuilder().setText(text).build();

        responseObserver.onNext(response);
        responseObserver.onCompleted();

    }

```

- Implements the **hello()** method from the `.proto` file.
- Reads `firstname` and `lastname` from the client request.
- Creates a response message: `"Hello World, <firstname> <lastname>"`.
- Sends the response back to the client with **onNext()**, then ends with **onCompleted()**.

Implementierung

Start HelloWorldServer (Java)

```

gradle clean build
gradle runServer

```

Start HelloWorldClient (Java)

```

gradle runClient

```

----- Python

Add grpcio packages

```

pip3 install grpcio grpcio-tools

```

Compile .proto file

```

python3 -m grpc_tools.protoc -I src/main/proto --python_out=src/main/resources --
grpc_python_out=src/main/resources src/main/proto/hello.proto

```

Start HelloWorldClient (Python)

```

python3 src/main/resources/helloWorldClient.py

```

Quellen
